Gauntlet

Error Analysis in LLM Evals

Finding What's Actually Broken

DEFINITION

What is error analysis?

> The process of a human **systematically reviewing LLM outputs** to identify where the model is going wrong.

- Not about metrics or scores — it's about human judgment and pattern recognition
- The foundation of any good eval system
- You look at traces, you notice things, you write them down

WHY FIRST?

Start here — before any eval infrastructure

Shapes your evals	Helps you decide what evals to write in the first place
Unique to you	Identifies failure modes specific to your application and data
Counterintuitive	Start here before building complex evaluation systems — fix obvious issues first

PROCESS

How to do error analysis

1. **Gather representative traces**
   Real user interactions; use synthetic data if you have none yet

2. **A domain expert reviews and journals**
   Open-ended notes — think of it as journaling, not scoring

3. **Focus on the first failure in each trace**
   Upstream errors cause downstream problems

4. **Categorize into a failure taxonomy**
   Group similar failures into distinct, named categories

5. **Iterate to theoretical saturation**
   Stop when new traces stop revealing new failure modes — aim for at least 100

TOOLING

Build your own lightweight tool

The problem with platforms

- Observability platforms aren't designed for your specific scenario
- Hard to customize what you see and how you see it
- You end up working around the tool

Build your own

- Access your trace logs directly
- Show exactly what matters for your use case
- Keeps the human reviewer in control
- Often just a simple script or small UI

PRACTICAL TIPS

Habits that keep error analysis honest

30-minute reviews	Manually review 20–50 outputs whenever you make significant changes
Read raw traces	Always do this yourself — never skip or delegate this step
Just fix obvious bugs	When you find a clear bug, go fix it — don't get nerd-sniped into building eval infrastructure around it
Classify by hand	A spreadsheet is often all you need to start

LLMS AS ASSISTANTS

LLMs can help — but can't replace you

Where LLMs help

- Categorizing and counting failures at scale
- Surfacing patterns across many traces
- First-pass grouping to speed up review

Where humans must stay in the loop

- LLM groupings conflate issues that need different fixes
- Criteria drift is real — your sense of "good" shifts as you review
- Error analysis is sensemaking, not a static target

> Error analysis is an **iterative, human-driven** process — not something you set and forget.

EVALUATION DESIGN

Binary first, scores second

SCORES	PASS / FAIL
Easy to game — optimize for a number	Forces clarity: "Did this achieve its goal?"
Require calibration before they mean anything	Grounded in real failure modes from error analysis
100% on an easy eval beats 70% on a hard one	A 70% rate on a tough eval beats 100% on an easy one

> Scores have their place — but only after you've grounded them in **real failure modes** you found in error analysis.

REALITY CHECK

How much time does this actually take?

> In practice: **60–80% of development time** goes to error analysis and evaluation.

Most work is human	Looking at data, not building automated checks
It's ongoing	Not a one-time step — revisit continuously as your system changes
The goal	Move from "vibe checking" to a systematic, data-driven improvement process

Key Takeaways

01 Error analysis is the most important activity in evals

02 Start with humans looking at traces — everything else follows from that

03 Build tools that serve your specific use case

04 Use LLMs as assistants, not decision-makers

05 Binary judgments first, scores second

06 Expect to spend most of your time here — that's normal and correct